

17. Сложност на алгоритми - сложност по време и сложност по памет. Асимптотични означения, класове на сложност. Анализ на сложността в най- добрия и най-лошия случай. Примери за класически алгоритми от класове n , n^2 , $n \log n$ и 2^n .

1. Въведение

Сложността на алгоритъм е **количествена мярка за ресурсите** (време, памет), които той изразходва в зависимост от размера на входните данни. Целта на анализа на сложността е да се получи мярка, която е независима от конкретната реализация и позволява да се сравняват алгоритми помежду им и да се прогнозира как ще се държат при по-големи входове.

Сложността обикновено се изразява като функция $T(n)$ на размера на входа , преброявайки броя елементарни операции, а не абсолютното време - по този начин анализът е приложим за всяка платформа.

2. Сложност по време (time complexity)

Времева сложност описва как расте броят операции (времето за изпълнение) на алгоритъма при увеличаване на размера на входа n . Тя се получава чрез:

1. Избор на елементарна операция (като сравнение, аритметична операция или обръщение към клетка от паметта)
2. Преброяване на колко пъти тази операция се изпълнява като функция на n .
3. Извеждане на затворена форма $T(n)$ или директно - асимптотичен запис, към който $T(n)$ принадлежи.

Например за цикъл, който обхожда масив от n елемента веднъж, броят сравнения е точно n ; следователно $T(n) = n$. За два вложени цикъла, всеки от които обхожда n елемента, броят операции е от порядъка на n^2 .

Времевата сложност рядко се дава като точна формула, обикновено интересува асимптотичното поведение при $n \rightarrow \infty$, защото за големи n константите и по-ниските членове на полинома стават несъществени спрямо доминиращия член.

3. Сложност по памет (space complexity)

Сложността по памет измерва количеството памет, което алгоритъмът използва като функция на n . Разделя се на 2 основни компонента:

- **Памет за входа** - паметта, необходима за самото представяне на входните данни. Обикновено не се брои в анализа, защото е една и съща за всеки алгоритъм над същия вход.
- **Спомагателна памет** - допълнителната памет, която алгоритъмът заделя в допълнение към входа: временните променливи, помощни масиви, стек на ресурсите и др. Именно това е паметта, върху която фокусираме анализа.

Алгоритъм, чиято спомагателна памет е константна $O(1)$ независимо от n , се нарича **алгоритъм на място**. Например **Insertion Sort** и **Bubble Sort** пренареждат елементите в самия входен масив, без да заделят допълнителна структура от порядъка на n .

Обратно, Merge Sort в класическата си реализация заделя допълнителен масив за сливането на всяка стъпка, което води до $\Theta(n)$ спомагателна памет.

При **рекурсивни** алгоритми паметта включва и **стека на извикванията** - всяко активно рекурсивно извикване заема място в стека, докато не завърши. Затова дълбочината на рекурсията пряко определя паметта:

- Рекурсивен линеен обход - дълбочина $O(n) \rightarrow O(n)$
- Балансирано разделяй-и-владей (като Merge Sort) - дълбочина $O(\log n) \rightarrow O(\log n)$ допълнителна памет само за стека + $\theta(n)$ за буфериране при сливане
- Наивна експоненциална рекурсия (напр. изчисляване на числа на Фибоначи) - дълбочината на стека е само $O(n)$, но общият брой извиквания е $\theta(2^n)$

4. Асимптотични означения

Асимптотичните означения формализират идеята за поведение при големи n , абстрахирайки се от константи и от поведението за малко n .

Стандартните означения са **пет**:

- O
- Ω
- Θ
- o
- ω

4.1 O - Асимптотична Горна Граница

$f(n) = O(g(n))$ означава, че $g(n)$ е горна граница за растежа на $f(n)$ с точност до константа:

$$O(g(n)) = f(n) : \exists c > 0, n_0 > 0 \text{ такива, че } 0 \leq f(n) \leq c \cdot g(n) \forall n \geq n_0$$

Използва се предимно за граница на **най-лошия случай** - гарантира, че алгоритъмът никога няма да "надрасне" тази граница. Скоростта на алгоритъма е по-бърза или равна на $g(n)$.

4.2 Ω -нотация - Асимптотична Долна Граница

$\Omega(g(n)) = f(n) : \exists c > 0, n_0 > 0$ такива, че $0 \leq c \cdot g(n) \leq f(n) \forall n \geq n_0$

Ω е точно обратното на O - вместо гарания, че $f(n)$ никога няма да надмине определен праг, тя гарантира, че $f(n)$ никога няма да падне под него.

Използва се за долна граница. Скоростта на алгоритъма е по-бавна или равна на $g(n)$.

4.3 Θ -нотация - Точна Асимптотична Граница

$f(n) = \Theta(g(n))$, тогава и само тогава, когато $f(n) = O(g(n))$ и $f(n) = \Omega(g(n))$.

Θ означава, че $f(n)$ е притисната отгоре и отдолу от $g(n)$ с точност до константи - $f(n) = O(g(n))$ и $f(n) = \Omega(g(n))$. Това е **най-силното** и **най-информативното** от петте означения, защото описва **точния порядък** на растеж, а не само граница.

4.4 o и ω нотации - Строги Граници

Малките нотации са много по-строги. Малкото o и малкото ω са строгите граници на O и Ω - изразяват се удобно чрез граница.

4.4.1 Малко o (Little o) - Строга горна граница

Алгоритъмът расте строго по-бавно от тази граница. Границата е недостижима.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

4.4.2 Малко ω (Little omega) - Строга долна граница

Алгоритъмът расте строго по-бързо от тази граница.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

4.5 Основни свойства

Основните свойства са:

- Транзитивност
- Рефлексивност
- Симетричност
- Транспонирана симетричност

Скорост на нарастване:

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$$

5. Класове на сложност

5.1 Именуване на основните класове

Означение	Име	Типичен алгоритъм
$O(1)$	Константна	достъп до елемент от масив по индекс
$O(\log n)$	Логаритмична	двоично търсене
$O(n)$	Линейна	линейно търсене, обхождане на масив
$O(n \log n)$	Линейно-логаритмична	Merge Sort, Heap Sort
$O(n^2)$	Квадратична	Insertion Sort, Bubble Sort
$O(n^3)$	Кубична	наивно умножение на матрици
$O(2^n)$	Експоненциална	генериране на всички подмножества
$O(n!)$	Факториелна	пълно изброяване на пермутации (напр. груб TSP)

Дори за умерено малко $n = 50$ разликата между n^2 (2 500 операции) и 2^n (над един квадрилон операции) е астрономическа. Това е практическото значение на класа на сложност: той определя дали алгоритъмът е приложим за реални размери на входа, независимо от константите и от скоростта на конкретния процесор.

6. Анализ на сложността в най-добрия случай и най-лошия случай

Времето за изпълнение на един алгоритъм не зависи единствено от количествения размер n , но и от конкретната конфигурация (структура и подредба) на входните данни. Затова се разграничават:

- Най-добър случай (best case) - минималният възможен брой операции за вход с размер n (най-благоприятното разположение на данните). Използва се рядко.
- Най-лош случай (worst case) - максималният възможен брой операции; дава гаранция, че алгоритъмът никога няма да работи по-бавно (затова е и най-често използваната мярка на практика).
- Среден случай (average case) - очакваният брой операции при определено вероятностно разпределение на входовете; изисква допускания за разпределението и обикновено е по-труден за извеждане математически.

7. Класически алгоритми и примери

7.1 Клас $\Theta(n)$ - линейни алгоритми

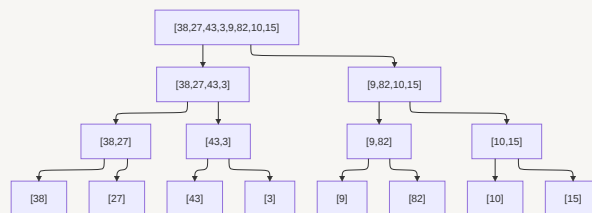
Линейното търсене обхожда масив елемент по елемент, докато намери търсената стойност или изчерпи масива:

```
for (int i = 0 ; i < size ; i++) {  
    if(a[i] == target) return i;  
}
```

- Най-добър случай: елементът е на първа позиция $\rightarrow O(1)$.
- Най-лош случай: елементът е на последна позиция или липсва $\rightarrow$ трябва да се провери всеки от n елемента $\rightarrow \Theta(n)$.
- Среден случай (елемент, равномерно разположен): около $n/2$ проверки $\rightarrow$ отново $\Theta(n)$.

7.2 Клас $\Theta(n \log n)$ - Merge Sort

Merge Sort е класически пример за парадигмата разделяй-и-владей: масивът се разделя рекурсивно на две половини, докато остане по един елемент.



Диаграмата на рекурсията има $\log_2 n$ нива (дървото се разполовява до достигане на единичен елемент); на всяко ниво общата работа за сливане на всички подмасиви е $\Theta(n)$ - оттук общо $\Theta(n) \cdot \Theta(\log n) = \Theta(n \log n)$.

Има $\Theta(n \log n)$ във всички случаи - най-добрия, най-лошия и средния, защото разделянето и сливането не зависят от конкретното разположение на елементите. Спомагателната памет е $\Theta(n)$ поради временния буфер при сливане.

7.3 Клас $\Theta(n^2)$ - Bubble sort

Bubble sort на всяка от $n - 1$ обхождания сравнява съседни елементи и ги разменя, ако са в грешен ред:

```

for (int i = 0; i < n - 1; ++i) {
    for (int j = 0; j < n - 1 - i; ++j) {
        if (a[j] > a[j + 1]) {
            std::swap(a[j], a[j + 1]);
        }
    }
}

```

Двата вложени цикъла дават общо приблизително $\frac{n(n-1)}{2}$ сравнения в най-лошия случай - $\Theta(n^2)$. Най-добрият случай пада до $\Theta(n)$ при допълнителна проверка.

7.4 Клас $\Theta(2^n)$ - експоненциални алгоритми

Този клас възниква естествено при алгоритми, които изследват всички възможни комбинации на n обекта или при наивна рекурсия с два неприпокриващи се извиквания на всяка стъпка.

Пример за такъв алгоритъм е:

Кулите на Ханой - за преместване на n диска от един стълб на друг, всяка рекурсивна стъпка премества $n - 1$ диска на помощен стълб, после един диск, после отново $n - 1$ диска обратно.

Наивна рекурсия за числа на Фибоначи.

```
long long fib(int n) {  
    if (n <= 1) return n;  
    return fib(n - 1) + fib(n - 2);    // два самостоятелни кл  
    она на рекурсията  
}
```

Рекурентното уравнение е $T(n) = T(n - 1) + T(n - 2) + \Theta(1)$. Горната граница е $T(n) = O(2^n)$.

7.5 Обобщителна таблица

Клас	Алгоритъм	Най-добър случай	Най-лош случай	Спомагателна памет
$\Theta(n)$	Линейно търсене	$O(1)$	$\Theta(n)$	$O(1)$
$\Theta(n \log n)$	Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
$\Theta(n^2)$	Bubble Sort	$\Theta(n)$	$\Theta(n^2)$	$O(1)$
$\Theta(2^n)$	Кули на Ханой или Фибоначи	$\Theta(2^n)$	$\Theta(2^n)$	$O(n)$ със стек